

Microsoft GraphRAG with an RDF Knowledge Graph - Part 2

Réécriture technique fidèle en français

D'après l'article de Ian Ormesher, publié sur Medium le 17 août 2024. Le document source annonce une lecture d'environ 4 minutes et porte sur le chargement des sorties de Microsoft GraphRAG dans un magasin RDF.

Important : ce PDF n'est pas une reproduction intégrale de l'article Medium. Il s'agit d'une reformulation technique originale, structurée pour le travail d'analyse GraphRAG/RDF, avec les limites explicitement indiquées.

Source principale	https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-2-d8d291a39ed1
Dépôt associé	https://github.com/ianormy/msft_graphrag_blog
Référence OWL 2	https://www.w3.org/TR/owl2-primer/

1. Objet de cette deuxième partie

Cette partie reprend le résultat attendu de la partie 1 : des fichiers Parquet générés par Microsoft GraphRAG. L'intention est de transformer ces sorties en données liées, exprimées en Turtle, puis de les charger dans un magasin RDF.

L'auteur ajoute une étape structurante avant le chargement des données : importer une ontologie décrivant le modèle GraphRAG. Cette ontologie sert de cadre explicite pour les classes et les relations qui seront ensuite peuplées par les données issues des fichiers Parquet.

La partie 2 prépare donc la partie 3 : une fois l'ontologie chargée, les données importées et un index vectoriel créé, le système sera prêt pour une expérimentation RAG combinant graphe RDF et recherche sémantique.

Les trois étapes annoncées

- Importer l'ontologie dans un magasin RDF.
- Créer et importer des données conformes à cette ontologie.
- Créer un index vectoriel qui servira à la recherche sémantique.

2. Chaîne de traitement reformulée

La logique technique peut être représentée ainsi :

Sorties GraphRAG en Parquet -> génération RDF/Turtle -> import dans GraphDB -> ontologie OWL dans le graphe par défaut -> index vectoriel Elasticsearch -> usage RAG dans la partie suivante

Cette chaîne illustre une séparation utile : le graphe RDF porte la structure sémantique et les relations, tandis que l'index vectoriel porte la similarité sémantique calculée sur les embeddings.

3. Importer l'ontologie dans un magasin RDF

Dans l'article, l'ontologie est présentée comme une description formelle des concepts d'un domaine et des relations entre ces concepts. Elle sert de base à un graphe de connaissances, car elle décrit explicitement ce que les données signifient et comment elles peuvent être reliées.

Le format mentionné est OWL 2, une recommandation du W3C pour exprimer des ontologies Web. L'auteur indique avoir créé une ontologie pour Microsoft GraphRAG avec Metaphactory, puis l'avoir exportée sous forme de fichier OWL.

Pour l'exécution pratique, l'auteur utilise GraphDB comme magasin RDF, en précisant qu'il existe une version gratuite. Il crée un dépôt vide nommé msft-graphrag, puis importe le fichier OWL fourni dans le dépôt.

Point opérationnel important : l'import doit se faire dans le graphe par défaut. Ce détail est explicitement mentionné dans l'article et évite de charger l'ontologie dans un graphe nommé que les étapes suivantes ne consulteraient pas nécessairement.

Pourquoi cette étape est importante

- Elle donne une structure explicite au graphe de connaissances.
- Elle aide à standardiser les relations et les types de ressources.
- Elle soutient la validation, l'explication et la vérification des données.
- Elle réduit le risque de construire un système opaque, difficile à justifier ou à auditer.

4. Créer et importer les données conformes à l'ontologie

Une fois l'ontologie chargée, les données peuvent être produites et importées. L'auteur indique avoir créé un notebook Jupyter qui lit les fichiers Parquet générés par la partie 1, puis produit un fichier Turtle correspondant.

Turtle est utilisé ici comme syntaxe RDF lisible par l'humain et importable dans un magasin RDF. Dans le scénario de l'article, ce fichier Turtle sert de pont entre les fichiers analytiques produits par GraphRAG et le graphe de connaissances RDF exploitable par requêtes et raisonnement sémantique.

L'auteur précise aussi qu'il fournit deux fichiers Turtle prêts à l'emploi : l'un basé sur des segments de taille 300, l'autre basé sur des segments de taille 1200 avec des données de corrélation. Cette précision vient de l'article ; le document ne détaille pas ici le contenu exact de ces fichiers.

Un autre notebook est mentionné pour analyser les données une fois qu'elles sont chargées dans le graphe de connaissances. Le dépôt GitHub associé contient les ressources de la série et indique que le dossier data contient notamment le fichier d'ontologie msft-graphrag.owl.

5. Créer un index vectoriel

La dernière étape décrite dans cette partie consiste à créer un index vectoriel avec Elasticsearch. L'objectif est de préparer une recherche sémantique utilisable avec le graphe RDF.

L'index concerne la classe Entity. Le champ stocké est `description_embedding`, présent dans le fichier Parquet des entités. L'association se fait avec le champ `id`, afin de relier les résultats vectoriels aux ressources correspondantes.

L'article ne présente pas encore la génération de réponse finale : il indique que cet index sera utilisé dans la partie 3, conjointement avec le graphe RDF, pour améliorer les réponses aux questions des utilisateurs.

6. Ce qui est confirmé par l'article

- La partie 2 s'appuie sur les fichiers Parquet produits dans la partie 1.
- Ces fichiers sont transformés en Turtle pour représenter des données liées.
- Une ontologie OWL dédiée à Microsoft GraphRAG est importée avant les données.
- GraphDB est utilisé comme magasin RDF dans la démonstration.
- Le dépôt GraphDB créé dans l'exemple est nommé msft-graphrag.
- L'import de l'ontologie doit être fait dans le graphe par défaut.
- Un notebook Jupyter sert à convertir les Parquet en Turtle.
- Deux fichiers Turtle prêts à l'emploi sont mentionnés : taille de segment 300, et taille de segment 1200 avec données de corrélation.
- Elasticsearch est utilisé pour créer un index vectoriel.
- L'index porte sur la classe Entity, avec le champ `description_embedding` relié au champ `id`.

7. Ce que l'article ne démontre pas encore

Pour rester strictement fidèle au contenu disponible, il faut distinguer les intentions annoncées des résultats démontrés dans cette partie.

- La partie 2 ne montre pas encore le pipeline de question-réponse complet ; cela est renvoyé à la partie 3.
- L'article ne fournit pas, dans son texte principal, une description détaillée de toutes les classes et propriétés de l'ontologie visible dans les captures.
- L'article ne compare pas GraphDB à Apache Jena Fuseki, Stardog, RDF4J ou d'autres magasins RDF.
- L'article ne présente pas de mesure de performance entre recherche vectorielle seule, graphe seul et combinaison graphe + vecteurs.
- L'article ne prétend pas que l'index vectoriel remplace le graphe RDF ; il le positionne comme un complément pour la recherche sémantique.

8. Lecture technique pour un projet GraphRAG RDF

L'intérêt principal de cette partie est de montrer comment déplacer les sorties de Microsoft GraphRAG vers un environnement RDF plus standardisé. Cette orientation est importante pour un projet fondé sur les technologies ouvertes du Web de données liées.

Dans une architecture GraphRAG classique, le graphe peut rester lié à un moteur ou à un format particulier. Ici, le passage par OWL, RDF et Turtle ouvre la voie à une représentation plus interopérable : les entités, relations, descriptions et corrélations peuvent être stockées dans un magasin RDF et interrogées avec des outils du Web sémantique.

Le rôle de l'index vectoriel reste complémentaire. Il sert à retrouver des entités proches d'une question ou d'un contexte utilisateur. Le graphe RDF, lui, sert à donner une structure, des relations explicites et un cadre d'explication. La combinaison des deux est le coeur de l'approche annoncée pour la partie 3.

9. Glossaire opérationnel

Terme	Définition dans le contexte de l'article
Parquet	Format de fichiers produit en sortie de la partie 1 et utilisé comme source de transformation.
Turtle	Syntaxe RDF utilisée pour écrire les données liées avant import dans le magasin RDF.
OWL	Langage d'ontologie utilisé pour formaliser les concepts et relations du domaine.
GraphDB	Magasin RDF utilisé dans la démonstration pour héberger l'ontologie et les données.
Entity	Classe ciblée par l'index vectoriel dans Elasticsearch.
description_embedding	Champ vectoriel provenant du fichier Parquet des entités et stocké dans l'index.

10. Checklist de reproduction

Cette checklist reformule l'enchaînement technique présenté dans l'article. Elle ne remplace pas les notebooks ni les fichiers fournis par l'auteur.

- Disposer des fichiers Parquet produits par la partie 1.
- Créer un dépôt vide dans GraphDB, nommé par exemple msft-graphrag.
- Importer le fichier OWL de l'ontologie dans le graphe par défaut.
- Générer un fichier Turtle à partir des fichiers Parquet avec le notebook Jupyter, ou utiliser l'un des fichiers Turtle fournis par l'auteur.

- Importer les données Turtle dans le magasin RDF.
- Créer un index Elasticsearch sur la classe Entity.
- Stocker le champ description_embedding et le relier au champ id.
- Utiliser ensuite, dans la partie 3, l'index vectoriel et le graphe RDF pour construire le mécanisme RAG.

11. Ressources citées

- Ian Ormesher, Microsoft GraphRAG with an RDF Knowledge Graph - Part 2, Medium, 17 août 2024 : <https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-2-d8d291a39ed1>
- Dépôt GitHub de Ian Ormesher, msft_graphrag_blog : https://github.com/ianormy/msft_graphrag_blog
- W3C, OWL 2 Web Ontology Language Primer : <https://www.w3.org/TR/owl2-primer/>
- Ressource mentionnée par l'article : travaux de Tomaz Bratanic sur Microsoft GraphRAG orientés Neo4j/Cypher.

Fin du document.